

JOBHUNTMANAGER

Railsバックエンド コード解説メモ

対象: 現在の backend ディレクトリ / Rails 8 API / PostgreSQL / Devise JWT

このメモの目的

コードを暗記するのではなく、リクエストがどのファイルを通り、どこで認証・認可・検証・JSON化されるかを理解するための教材です。実際のコード抜粋と「なぜ」をセットにしています。

想定読者: Rails初心者から中級者 / 作成日: 2026-06-21

参照範囲: Gemfile、config、migration、model、controller、serializer、test、GitHub Actions

最初に覚える3つの視点

視点	見る場所	考えること
データ	migration / model	何を保存し、どんな値を禁止するか
リクエスト	routes / controller	誰が、どのデータへ、何を実行できるか
レスポンス	serializer / tests	Reactへ何を返し、仕様をどう保証するか

Rails APIを読むコツ

1ファイルずつ孤立して読むより、URLからroutesを探し、controllerのaction、model、serializer、testへ進むと理解しやすくなります。

目次

- 1. 全体アーキテクチャとリクエストの流れ
- 2. Gemfile・Rails API設定・環境変数
- 3. routes.rbとRESTful設計
- 4. データベース・migration・関連図
- 5. Model: 関連付け・Validation・enum
- 6. Devise JWT認証
- 7. 共通Controllerとエラー形式
- 8. 業務API Controller
- 9. SerializerとN+1対策
- 10. テストコードの読み方
- 11. 代表的なリクエストを追跡する
- 12. よく使うRails構文と次に学ぶこと

1. 全体アーキテクチャとリクエストの流れ

このプロジェクトはRailsがHTMLを作らず、React SPAへJSONだけを返すAPI専用構成です。Railsの責務は、認証、データ所有者の確認、入力検証、DB操作、JSONレスポンスの生成です。

順番	担当	このプロジェクトでの例
1	React	Authorization: Bearer JWTを付けてAPIを呼ぶ
2	routes.rb	URLとHTTPメソッドをControllerのactionへ振り分ける
3	BaseController	JWTからcurrent_userを決定し、未認証なら401
4	各Controller	current_userの関連からデータを取得し、Strong Parametersで入力を制限
5	Model	関連付け、enum、Validation、削除ルールを適用
6	PostgreSQL	外部キー、NOT NULL、UNIQUE制約で最終防衛
7	Serializer	Reactが使う形のHashを作る
8	Controller	HTTPステータスとJSONを返す

重要: 認証と認可は別物

認証は「このJWTは誰か」を決めることです。認可は「その人がこの求人や応募を操作してよいか」を決めることです。BaseControllerが認証、current_user.job_postings.find(...)のような検索が認可を担います。

MVCにSerializerを足した構成

層	主な責務	置き場所
Model	DBとの対応、関連、業務ルール、Validation	app/models
Controller	HTTP入力、認証・認可、処理順序、ステータス	app/controllers
Serializer	外部へ公開するJSON項目とネスト構造	app/serializers
Migration	DB構造の変更履歴	db/migrate
Integration Test	APIを入口から出口まで検証	test/integration

2. Gemfile ・ Rails API設定 ・ 環境変数

Gemfile: 必要な機能をGemとして宣言する

backend/Gemfile

```
gem "rails", "~> 8.0.4"
gem "pg", "~> 1.1"
gem "rack-cors"
gem "puma", ">= 5.0"

gem "devise", "~> 5.0"
gem "devise-jwt", "~> 0.13.0"

group :development, :test do
  gem "brakeman", require: false
  gem "rubocop-rails-omakase", require: false
  gem "dotenv-rails"
end
```

- rails: MVC、Routing、Active Record、Controllerなどアプリの土台です。
- pg: Ruby/RailsからPostgreSQLへ接続するドライバです。
- rack-cors: localhost:5173のReactからlocalhost:3000のRailsへ通信できるようにします。
- devise / devise-jwt: パスワード認証とJWT発行・失効を担当します。
- brakeman / rubocop: セキュリティとコードスタイルをCIで検査します。

API専用モード

backend/config/application.rb

```
module Backend
  class Application < Rails::Application
    config.load_defaults 8.0
    config.api_only = true
  end
end
```

config.api_only = trueにより、HTML View、Cookie Session、Flashなど通常のRails画面向け機能を減らします。Reactが画面を作るため、Rails側はJSON APIに集中できます。

環境変数とデータベース

backend/config/database.yml (抜粋)

```
default: &default
  adapter: postgresql
  encoding: unicode
  pool: <%= ENV.fetch("RAILS_MAX_THREADS") { 5 } %>
  username: <%= ENV["DB_USERNAME"] %>
  password: <%= ENV["DB_PASSWORD"] %>
  host: localhost
```

backend/.env.example

```
DB_USERNAME=postgres
DB_PASSWORD=your_database_password
DEVISE_JWT_SECRET_KEY=replace_with_a_long_random_secret
FRONTEND_ORIGIN=http://localhost:5173
```

パスワードやJWT秘密鍵をコードへ直書きしない理由は、GitHubへ漏れると第三者がDBへ接続したりJWTを偽造できるからです。.env.exampleは「必要な変数名だけ」を共有する見本で、実値は各環境に置きます。

秘密情報の扱い

DEVISE_JWT_SECRET_KEYは十分長いランダム値にし、本番・開発・テストで分けます。.env本体はGit管理しません。

CORS: ReactにAuthorizationヘッダーを見せる

backend/config/initializers/cors.rb

```
Rails.application.config.middleware.insert_before 0, Rack::Cors do
  allow do
    origins ENV.fetch("FRONTEND_ORIGIN", "http://localhost:5173")

    resource "**",
      headers: %w[Authorization Content-Type Accept],
      methods: [ :get, :post, :put, :patch, :delete, :options, :head ],
      expose: %w[Authorization]
    end
  end
end
```

originsは通信を許可するフロントエンドです。expose: Authorizationが重要で、これがないとブラウザ上のReactはクインレスポンスに付いたJWTヘッダーをJavaScriptから読めません。

3. routes.rbとRESTful設計

backend/config/routes.rb (業務API)

```
namespace :api do
  namespace :v1 do
    resources :companies, only: [:index create]
    resources :job_postings
    resources :applications do
      resources :interviews, only: :create
      resources :tasks, only: :create
      resources :notes, only: [:index create]
    end
    resources :interviews, only: [:index update destroy]
    resources :tasks, only: [:index update destroy]
    resources :notes, only: [:update destroy]
    resource :kanban, only: :show, controller: "kanban"
    patch "applications/:application_id/status",
      to: "application_statuses#update"
  end
end
```

namespace :api / :v1はURLを/api/v1にし、将来v2を追加しても既存Reactを壊しにくくします。resourcesはRESTfulなURLとController actionをまとめて生成します。

HTTP	URL例	Action	意味
GET	/api/v1/job_postings	index	一覧取得
GET	/api/v1/job_postings/10	show	1件取得
POST	/api/v1/job_postings	create	新規作成
PATCH	/api/v1/job_postings/10	update	部分更新
DELETE	/api/v1/job_postings/10	destroy	削除

なぜ子リソースをネストするのか

面接作成はPOST /applications/:application_id/interviewsです。「どの応募に面接を追加するか」がURLから明確になります。一方、更新時は面接IDだけで対象が一意なのでPATCH /interviews/:idです。

resourceとresourcesの違い

kanbanはユーザーごとに1つの画面データとして扱うため単数形resourceを使い、GET /api/v1/kanbanになります。IDをURLへ含めません。

4. データベース・migration・関連図

モデル関連

親	関係	子	削除時
User	1対多	Company	子があれば拒否
User	1対多	JobPosting	子があれば拒否
User	1対多	Application	子があれば拒否
Company	1対多	JobPosting	求人があれば拒否
JobPosting	1対0..1	Application	応募があれば拒否
Application	1対多	Interview / Task / Note	まとめて連鎖削除

Interview、Task、Noteにはuser_idがありません。所有者はapplication.user_idを通して判定します。重複したuser_idを持たせないことで、application.user_idとchild.user_idが食い違う矛盾を防ぎます。

t.referencesと外部キー

create_interviews migration（抜粋）

```
t.references :application,
  null: false,
  foreign_key: { on_delete: :cascade }

add_index :interviews, :scheduled_at
```

- null: false: 応募に属さない面接をDBレベルで禁止します。
- foreign_key: 存在しないapplication_idを保存できません。
- on_delete: :cascade: 応募削除時に子レコードもDB側で削除します。
- index: 面接日時による一覧・並び替えを速くするための索引です。

restrictとcascadeの使い分け

migrationの削除制約

```
# 企業や求人は履歴を守る
foreign_key: { on_delete: :restrict }

# 応募を消すと面接・タスク・メモも消す
foreign_key: { on_delete: :cascade }
```

企業や求人を先に消すと応募履歴の意味が失われるためrestrictです。応募自体を確認して削除する場合は、その応募専用の面接・タスク・メモを残す意味が薄いためcascadeです。

JTI追加Migrationがup/downを使う理由

20260619000001_add_jti_to_users.rb（抜粋）

```
def up
  add_column :users, :jti, :string

  MigrationUser.reset_column_information
  MigrationUser.find_each do |user|
    user.update_columns(jti: SecureRandom.uuid)
  end

  change_column_null :users, :jti, false
  add_index :users, :jti, unique: true
end
```

既存ユーザーがいる状態で、いきなりNOT NULL列を追加するとMigrationが失敗します。先にNULL許可で列を追加し、既存行へUUIDを埋め、その後NOT NULLとUNIQUEを付けます。段階的変更が必要なのでchangeではなくup/downです。

Model Validationだけでは不十分

同時リクエストやRails以外からのDB操作ではValidationをすり抜ける可能性があります。重要な一意性・外部キー・NULL禁止はDB制約でも守ります。

5. Model: 関連付け・Validation・enum

User: 認証の中心と所有データ

backend/app/models/user.rb

```
class User < ApplicationRecord
  include Devise::JWT::RevocationStrategies::JTIMatcher

  devise :database_authenticatable, :registerable,
         :recoverable, :rememberable, :validatable,
         :jwt_authenticatable, jwt_revocation_strategy: self

  has_many :companies, dependent: :restrict_with_error
  has_many :job_postings, dependent: :restrict_with_error
  has_many :applications, dependent: :restrict_with_error

  validates :name, presence: true, length: { maximum: 255 }
end
```

has_manyによりcurrent_user.companiesのような関連検索が使えます。restrict_with_errorは子データがあるユーザー削除を失敗させ、エラーをModelへ追加します。JTIMatcherはusers.jtiとJWT内のjtiが一致するかを確認します。

Company / JobPosting: 入力制限と所有者整合性

backend/app/models/job_posting.rb (抜粋)

```
belongs_to :user
belongs_to :company
has_one :application, dependent: :restrict_with_error

validates :title, presence: true, length: { maximum: 255 }
validates :source_url,
  format: { with: URL_FORMAT },
  length: { maximum: 2_048 },
  allow_blank: true
validate :company_must_belong_to_user

def company_must_belong_to_user
  return if company.blank? || user.blank?
  return if company.user_id == user_id

  errors.add(:company, "はログインユーザーが所有する企業を指定してください")
end
```

Controllerでもcurrent_user.companies.findを使いますが、Modelにも同一ユーザー確認を置いています。Rails console、別Controller、将来のバッチ処理から作成されても不正な組み合わせを保存しないためです。

Application: カンバンの中心

backend/app/models/application.rb

```

belongs_to :user
belongs_to :job_posting

has_many :interviews, dependent: :destroy
has_many :tasks, dependent: :destroy
has_many :notes, dependent: :destroy

enum :status, {
  applied: 0,
  document_screening: 1,
  interview_scheduled: 2,
  offered: 3,
  rejected: 4
}, validate: true

validates :applied_on, presence: true
validates :job_posting_id, uniqueness: true

```

DBにはstatusを整数で保存し、APIではapplication.statusが'applied'のような文字列になります。enumにより検索用scopeや判定メソッドも生成されます。validate: trueにより未知のstatusを422として扱えます。

一求人一応募

job_posting_idのModel ValidationとDBのUNIQUE indexを両方使っています。通常の入力エラーと、同時実行による競合の両方へ備える設計です。

Interview: 複数enumにprefixを付ける

backend/app/models/interview.rb (抜粋)

```

enum :interview_type, {
  casual: 0, first: 1, second: 2, final: 3, other: 4
}, prefix: true, validate: true

enum :status, {
  scheduled: 0, completed: 1, canceled: 2
}, prefix: true, validate: true

enum :result, {
  pending: 0, passed: 1, failed: 2
}, prefix: true, validate: true

```

複数enumにcompletedやpendingなど似た名前があるとメソッド名が衝突しやすいためprefixを使います。例: interview.status_completed?、interview.result_passed?。

Task: 型変換前の値を検証する

backend/app/models/task.rb

```

validate :due_at_must_be_valid

def due_at_must_be_valid
  raw_due_at = due_at_before_type_cast
  return if raw_due_at.blank? || due_at.present?

  errors.add(:due_at, "は正しい日時を指定してください")
end

```

Active Recordは'invalid-date'をdatetimeへ変換できないとnilにすることがあります。due_atだけを見ると、未入力nilと不正入力nilを区別できません。before_type_castで送信された元の値を見て、不正形式を422にします。

Note: API名contentとDB名bodyを橋渡しする

backend/app/models/note.rb

```
class Note < ApplicationRecord
  belongs_to :application

  alias_attribute :content, :body

  validates :content, presence: true, length: { maximum: 10_000 }
end
```

DBカラムを変更せず、APIではcontentという分かりやすい名称に統一しています。alias_attributeによりnote.contentの読み書きが実際にはbodyへ接続され、ValidationやStrong Parametersもcontentで書けます。

6. Devise JWT認証

JWT設定

backend/config/initializers/devise.rb (抜粋)

```
config.skip_session_storage = %i[http_auth params_auth]

config.jwt do |jwt|
  jwt.secret = ENV.fetch("DEVISE_JWT_SECRET_KEY")
  jwt.dispatch_requests = [
    [ "POST", %r{\A/api/v1/auth(?:\.json)?\z} ],
    [ "POST", %r{\A/api/v1/auth/sign_in(?:\.json)?\z} ]
  ]
  jwt.revocation_requests = [
    [ "DELETE", %r{\A/api/v1/auth/sign_out(?:\.json)?\z} ]
  ]
  jwt.expiration_time = 1.day.to_i
end
```

- skip_session_storage: Cookie Sessionではなく Bearer JWTだけを使います。
- dispatch_requests: 登録・ログイン成功時にJWTをAuthorizationレスポンスヘッダーへ載せます。
- revocation_requests: ログアウト時にusers.jtiを更新し、古いJWTを無効化します。
- ENV.fetch: 秘密鍵が未設定なら起動を失敗させ、危険な既定値で動くことを防ぎます。

ユーザー登録

registrations_controller.rb (抜粋)

```
def create
  build_resource(sign_up_params)

  if resource.save
    sign_in(resource_name, resource, store: false)
    render json: {
      data: UserSerializer.new(resource).serializable_hash
    }, status: :created
  else
    clean_up_passwords(resource)
    render_validation_errors(resource)
  end
end

def sign_up_params
  params.require(:user).permit(
    :name, :email, :password, :password_confirmation
  )
end
```

Devise::RegistrationsControllerを継承し、Deviseのユーザー構築・認証処理を利用しつつJSON形式だけを独自化しています。store: falseはサーバーSessionへ保存しない指定です。

ログイン・me・ログアウト

sessions_controller.rb (抜粋)

```
def create
  self.resource = warden.authenticate(auth_options)

  unless resource
    return render json: {
      error: {
        code: "invalid_credentials",
        message: "メールアドレスまたはパスワードが正しくありません"
      }
    }, status: :unauthorized
  end

  sign_in(resource_name, resource, store: false)
  render json: {
    data: UserSerializer.new(resource).serializable_hash
  }, status: :ok
end

def show
  render json: {
    data: UserSerializer.new(current_user).serializable_hash
  }, status: :ok
end

def destroy
  head :no_content
end
```

warden.authenticateがメールアドレスとパスワードを確認します。JWT自体のヘッダー追加・失効はdevise-jwtのMiddleware/Hookが担当するため、ControllerではJWT文字列を手作業で生成していません。

ログアウトが空に見える理由

destroyはhead :no_contentですが、リクエストがrevocation_requestsの正規表現に一致するとdevise-jwtがJTIを更新します。Controllerだけでなく設定とGemの動作を合わせて読む必要があります。

7. 共通Controllerとエラー形式

ApplicationController: 全APIの共通部品

backend/app/controllers/application_controller.rb

```
class ApplicationController < ActionController::API
  rescue_from ActionController::ParameterMissing,
    with: :render_bad_request
  rescue_from ActiveRecord::RecordNotFound,
    with: :render_not_found

  protected

  def render_validation_errors(record)
    render json: {
      error: {
        code: "validation_error",
        message: "入力内容を確認してください",
        details: record.errors.to_hash(true)
      }
    }, status: :unprocessable_entity
  end
end
```

rescue_fromで例外を共通JSONへ変換します。各Controllerに同じrescueを書く必要がなく、Reactもerror.code / message / detailsという一定の形で扱えます。

状況	HTTP	code	発生例
ルートキー不足	400	bad_request	params.require(:company)が失敗
JWTなし・無効	401	unauthorized	current_userが存在しない
対象なし・他人のデータ	404	not_found	current_user配下のfindが失敗
Validation失敗	422	validation_error	必須、文字数、enum不正
子データあり	422	dependent_exists	応募付き求人への削除

BaseController: 業務APIをまとめて認証必須にする

backend/app/controllers/api/v1/base_controller.rb

```
module Api
  module V1
    class BaseController < ApplicationController
      before_action :authenticate_user_with_json!
    end
  end
end
```

CompaniesControllerなどがBaseControllerを継承するだけで、action実行前にJWT認証が走ります。認証を書き忘れるリスクを減らすための継承設計です。認証APIはDevise Controllerを直接継承するため、ログイン前にもアクセスできます。

Strong Parameters

company_params

```
def company_params
  params.require(:company).permit(
    :name, :website_url, :industry, :location, :description
  )
end
```

requireはJSONにcompanyキーを要求し、なければ400です。permitは更新可能な項目だけを通します。user_idをpermitしないため、攻撃者が他ユーザーIDを送っても無視されます。

8. 業務API Controller

Companies: current_userからbuildする

companies_controller.rb (抜粋)

```
def index
  companies = current_user.companies.order(:name, :id)
  render json: {
    data: companies.map do |company|
      CompanySerializer.new(company, summary: true).serializable_hash
    end
  }, status: :ok
end

def create
  company = current_user.companies.build(company_params)
  if company.save
    render json: {
      data: CompanySerializer.new(company).serializable_hash
    }, status: :created
  else
    render_validation_errors(company)
  end
end
```

Company.new(company_params)ではなくcurrent_user.companies.buildを使うことで、user_idをリクエストから受け取らず、必ずログインユーザーを所有者にできます。

JobPostings: 他ユーザーのcompany_idを404にする

job_postings_controller.rb (抜粋)

```
def create
  company = find_company(job_posting_params[:company_id])
  job_posting = current_user.job_postings.build(
    job_posting_attributes.merge(company: company)
  )
  # save and render...
end

def find_company(company_id)
  return if company_id.blank?

  current_user.companies.find(company_id)
end
```

Company.find(company_id)だと全ユーザーの企業を検索してしまいます。current_user.companies.findなら、自分の企業でなければRecordNotFoundとなり、共通404へ変換されます。存在の有無も他ユーザーへ漏らしにくい設計です。

更新時のcompany_id nilを無視しない工夫

find_company_for_update

```
def find_company_for_update
  return @job_posting.company unless job_posting_params.key?(:company_id)

  find_company(job_posting_params[:company_id])
end
```

key?は「値がnilか」ではなく「キーが送られたか」を判定します。company_idが未送信なら既存値を維持し、company_id: nullが明示されたらcompany=nilを代入してbelongs_toのValidationで422にします。

Applications: 作成時だけ求人を選び、所有権を固定する

applications_controller.rb (抜粋)

```
def create
  job_posting = find_job_posting(
    create_application_params[:job_posting_id]
  )
  application = current_user.applications.build(
    create_application_params
    .except(:job_posting_id)
    .merge(job_posting: job_posting)
  )
  # save...
  rescue ActiveRecord::RecordNotUnique
    application.errors.add(:job_posting_id, :taken)
    render_validation_errors(application)
  end

  def update_application_params
    params.require(:application).permit(:status, :applied_on)
  end
end
```

job_posting_idはcreate用Parametersだけで許可し、updateでは許可しません。応募後に別求人へ付け替えると面接・タスク・メモの意味まで変わるためです。RecordNotUniqueは同時リクエストでDBのUNIQUE制約に負けた場合を422へ直します。

現在のコード上の注意

通常のApplications更新でもstatusをpermitしています。カンバン専用APIだけでstatusを変える方針へ厳密に合わせる場合は、update_application_paramsを:applied_onだけにする余地があります。本メモは現在の実装を解説しています。

Interview / Task / Note: application経由で所有者を判定

interviews_controller.rb (抜粋)

```
def set_application
  @application = current_user.applications.find(params[:application_id])
end

def set_interview
  @interview = Interview
    .joins(:application)
    .where(applications: { user_id: current_user.id })
    .find(params[:id])
end
```

子テーブルにuser_idがないため、applicationsをJOINして所有者を絞ります。Interview.find(params[:id])だけでは他ユーザーの面接を更新できてしまいます。URLのIDを信用せず、常にcurrent_user条件をSQLへ入れるのが重要です。

Taskのcompletedは仮想的なAPI入力

tasks_controller.rb (抜粋)

```
def assign_completion(task)
  return true unless task_params.key?(:completed)

  completed = task_params[:completed]
  unless [true, false].include?(completed)
    task.errors.add(:completed, "はtrueまたはfalseを指定してください")
    return false
  end

  task.completed_at = completed ? Time.current : nil
  true
end
```

DBにはboolean completed列ではなく completed_atがあります。APIは扱いやすいtrue/falseを受け取り、trueなら完了日時、falseならnilへ変換します。完了時刻が残るため、後から「いつ完了したか」も表示できます。

Kanban: 全列を必ず返す

kanban_controller.rb

```
applications = current_user
  .applications
  .includes(job_posting: :company)
  .order(:status, updated_at: :desc, id: :desc)

grouped_applications = Application.statuses.keys.index_with { [] }
applications.each do |application|
  grouped_applications.fetch(application.status) <<
    KanbanCardSerializer.new(application).serializable_hash
end
```

最初に全statusを空配列で用意するため、応募が0件の列も必ず返ります。Reactはキーの有無を毎回確認せず5列を描画できます。updated_at DESC、同時刻id DESCにより移動したカードが列の先頭へ来ます。

Status専用Controller

application_statuses_controller.rb

```
def update
  if @application.update(status_params)
    render json: {
      data: KanbanCardSerializer.new(@application).serializable_hash
    }, status: :ok
  else
    render_validation_errors(@application)
  end
end

def status_params
  params.require(:application).permit(:status)
end
```

ドラッグ&ドロップはstatusだけを変える操作なので、専用EndpointとStrong Parametersを作っています。applied_onなどを混ぜて変更できないため、意図が明確で安全です。

9. SerializerとN+1対策

なぜSerializerが必要か

Modelをそのままrenderすると、将来追加した内部カラムが意図せずAPIへ出る可能性があります。Serializerで公開項目を明示すると、DB構造とAPI仕様を分離できます。

user_serializer.rb

```
class UserSerializer
  def initialize(user)
    @user = user
  end

  def serializable_hash
    {
      id: user.id,
      name: user.name,
      email: user.email
    }
  end
end
```

encrypted_passwordやjtiを絶対に返さず、Reactに必要な3項目だけを返しています。

summaryとdetailを分ける

company_serializer.rb (抜粋)

```
def serializable_hash
  return summary_hash if summary

  summary_hash.merge(
    website_url: company.website_url,
    industry: company.industry,
    location: company.location,
    description: company.description,
    created_at: company.created_at,
    updated_at: company.updated_at
  )
end
```

企業選択一覧ではid/nameだけ、作成後レスポンスでは詳細も返します。用途に応じてJSONを小さくし、同じ組み立てコードを再利用します。

N+1問題とincludes

job_postings_controller.rb index

```
job_postings = current_user
  .job_postings
  .includes(:company, :application)
  .order(created_at: :desc, id: :desc)
```

求人10件を取得した後、Serializerが各求人人のcompanyとapplicationを読むと、事前読み込みなしでは追加SQLが最大20回発生します。includesで関連をまとめて読み、件数が増えてもSQL回数が増えにくくします。

ControllerとSerializerはセット

Serializerがどの関連を読むかを見て、Controllerがincludesを用意します。Serializerだけ、Controllerだけを見てもN+1対策は判断できません。

TaskSerializerの計算フィールド

task_serializer.rb (抜粋)

```
def overdue?  
  task.due_at.present? &&  
    task.completed_at.blank? &&  
    task.due_at < Time.current  
end
```

overdueは保存値ではなく、現在時刻で変化する派生情報です。DBへ保存すると時間経過で古くなるため、レスポンス生成時に計算します。

Application詳細のネスト

ApplicationSerializerはdetail: trueのときだけinterviews、tasks、notesを含めます。一覧APIで重い子データを返さず、詳細画面だけに必要な情報をまとめる設計です。set_application側で全関連をincludesしているため、Serializer内のアクセスもN+1になりません。

10. テストコードの読み方

Integration TestはAPI利用者の視点

authentication_test.rb (抜粋)

```
jwt = sign_in_and_fetch_jwt(users(:one))
headers = bearer_headers(jwt)

get "/api/v1/auth/me", headers: headers
assert_response :ok

delete "/api/v1/auth/sign_out", headers: headers
assert_response :no_content

get "/api/v1/auth/me", headers: headers
assert_response :unauthorized
```

ログインで本物のJWTを受け取り、そのJWTでme、ログアウト、再度meまで確認しています。メソッド単体ではなく、Devise設定・Router・Controller・JTI失効が連動していることを保証します。

認可テスト

job_postings_test.rb (考え方)

```
other_company = create_company(users(:two))

post "/api/v1/job_postings",
  params: {
    job_posting: {
      company_id: other_company.id,
      title: "不正な求人"
    }
  },
  headers: authorization_headers(users(:one)),
  as: :json

assert_response :not_found
assert_no_difference("JobPosting.count")
```

ステータスが404であることだけでなく、DB件数が増えていないことも確認します。HTTPレスポンスと副作用を両方検証するのが重要です。

N+1をSQL回数で検査する

select_query_count (共通パターン)

```
first_query_count = select_query_count do
  get "/api/v1/kanban", headers: authorization_headers(users(:one))
end

# データを増やす
multiple_query_count = select_query_count do
  get "/api/v1/kanban", headers: authorization_headers(users(:one))
end

assert_equal first_query_count, multiple_query_count
```

1件から3件へ増やしてもSELECT回数が同じなら、レコードごとの追加SQLが起きていないと判断できます。includesを削除したときにテストが失敗する安全網です。

失敗しにくい処理をstubで強制する

applications_test.rb (削除失敗)

```
with_stubbed_instance_method(
  Application,
  :destroy,
  lambda {
    errors.add(:base, "応募を削除できません")
    false
  }
) do
  delete "/api/v1/applications/#{application.id}",
    headers: authorization_headers(users(:one))
end

assert_response :unprocessable_entity
```

通常はdestroyが成功しやすいため、メソッドを一時的に差し替えて失敗経路を実行します。ensureで元へ戻している
ので、後続テストへ影響を残しません。

CI: GitHub上でも同じ品質確認

Job	コマンド	目的
scan_ruby	bundle exec brakeman --no-pager	Rails特有の脆弱性を静的解析
lint	bundle exec rubocop -f github	Rubyの書式・読みやすさを確認
test	rails db:test:prepare test	PostgreSQLを起動して全テスト

11. 代表的なリクエストを追跡する

例1: 求人を新規作成する

段階	コード	なぜ
Route	POST /api/v1/job_postings → create	HTTP操作をactionへ対応
認証	BaseController before_action	JWTなしなら処理前に401
入力	job_posting_params	user_idなど不要項目を遮断
認可	current_user.companies.find	他ユーザー企業を指定させない
所有者	current_user.job_postings.build	user_idをサーバー側で確定
検証	JobPosting Validation	必須、URL、文字数、所有者整合性
保存	save	Validation後にPostgreSQLへINSERT
出力	JobPostingSerializer	公開項目だけJSON化
応答	201 Created	作成成功をHTTPとして表現

例2: カンバンカードを内定列へ移動する

段階	処理
React	PATCH /applications/20/statusへ{ application: { status: 'offered' } }
Route	ApplicationStatusesController#update
認証	JWTからcurrent_userを復元
認可	current_user.applications.find(20)
入力制限	permit(:status)のみ
Model	enum validate: trueでofferedを許可
DB	statusとupdated_atを更新
Serializer	会社名・求人名を含むKanbanCardを返す
次回一覧	updated_at DESCのため移動カードが列先頭

例3: 他ユーザーのタスクを更新しようとする

Task.find(id)ではなく、Task.joins(:application).where(applications: { user_id: current_user.id }).find(id)です。対象IDが実在しても、自分のapplicationに属さなければ検索結果は0件となりRecordNotFoundです。ApplicationControllerが404 JSONへ変換し、更新処理には到達しません。

セキュリティの基本

フロントエンドでボタンを隠すだけでは認可になりません。攻撃者はcurlなどで任意IDを送れます。必ずRails側のSQL条件で所有者を限定します。

12. よく使うRails構文と次に学ぶこと

このプロジェクトで頻出する構文

構文	意味
<code>before_action :set_x</code>	action前に共通の検索処理を実行
<code>params.require(:x).permit(...)</code>	JSONルートキーと更新可能項目を制限
<code>current_user.x.build</code>	所有者をログインユーザーへ固定して新規構築
<code>includes(...)</code>	関連を事前読み込みしてN+1を防ぐ
<code>joins(...).where(...)</code>	関連テーブルの条件をSQLへ入れる
<code>render json:, status:</code>	JSON本文とHTTPステータスを返す
<code>head :no_content</code>	本文なしの204を返す
<code>rescue_from</code>	特定例外を共通処理へ変換
<code>dependent: :destroy</code>	親削除時にRailsから子も削除
<code>restrict_with_error</code>	子がある親の削除を拒否してerrorsへ追加
<code>enum ..., validate: true</code>	整数保存と文字列API、未知値のValidation
<code>alias_attribute</code>	DB列へ別名のRubyアクセサを付ける

実装を変更するときの確認順

- ・仕様書: 変更したい動作とHTTPステータスを確認する
- ・Migration/Schema: 新しい保存項目や制約が必要か
- ・Model: 業務ルールとValidationはどこへ置くか
- ・Route/Controller: 認証・認可・Strong Parametersは十分か
- ・Serializer: Reactへ必要な項目だけ返しているか
- ・Test: 正常系だけでなく他ユーザー、不正値、失敗時も追加する
- ・rails test / rubocop / brakeman / zeitwerk:checkを実行する

ローカル確認コマンド

PowerShell

```
cd D:\RubyProjects\job-hunt-manager\backend

bundle install
bundle exec rails db:migrate
bundle exec rails db:test:prepare
bundle exec rails test
bundle exec rubocop
bundle exec brakeman --no-pager
bundle exec rails zeitwerk:check
```

次に学ぶと理解が深まるテーマ

- ・Active Recordが生成するSQLをRailsログで読む

- トランザクションと同時実行、UNIQUE制約の競合
- JWTの構造（header / payload / signature）とJTI失効
- Rack MiddlewareとDevise/Wardenのリクエスト処理
- Serializer設計とAPIの後方互換性
- request spec / integration testで境界値と認可を設計する

最後のまとめ

このバックエンドの核は、(1) current_userの関連から検索して認可する、(2) Strong Parametersで変更可能項目を絞る、(3) ModelとDB制約の二重でデータを守る、(4) SerializerでReact向けJSONを固定する、(5) Integration Testで全体を保証する、の5点です。

このPDFは2026-06-21時点のD:\RubyProjects\job-hunt-manager\backendの実装を基に作成しています。